

Artificial Intelligence and Modern Application Development

Artificial Intelligence is a field of computer science that focuses on creating systems that can perform tasks that normally require human intelligence. These tasks include understanding natural language, recognizing images, making predictions, solving problems, and generating content. Modern AI applications are commonly built using programming languages such as Python because Python provides many useful libraries and frameworks for machine learning, data processing, and application development.

Python is a high-level programming language that is widely used in artificial intelligence, machine learning, data science, automation, and backend development. Libraries such as NumPy and Pandas are commonly used for data processing, while Scikit-learn provides tools for traditional machine learning algorithms. Python is also widely used for building APIs and integrating AI models into real-world applications.

FastAPI is a modern Python web framework used to build APIs. It is designed to be fast, easy to use, and suitable for developing backend services. FastAPI uses Python type hints and provides automatic API documentation. Developers can use FastAPI to create endpoints that receive user requests, process data, communicate with databases or AI models, and return responses. Because of its performance and simplicity, FastAPI is commonly used in modern Python backend and AI applications.

A REST API allows different software applications to communicate with each other over HTTP. Common HTTP methods include GET, POST, PUT, and DELETE. A GET request is generally used to retrieve information, while a POST request is commonly used to send data to a server. REST APIs are important in AI applications because an AI model can be exposed through an API and then used by websites, mobile applications, or other software systems.

Generative AI is a type of artificial intelligence that can generate new content based on a user's input. The generated content can include text, images, code, audio, or other types of data. Large Language Models, commonly called LLMs, are a major part of modern Generative AI systems. LLMs are trained on large amounts of text and can understand and generate human-like language.

Retrieval Augmented Generation, commonly called RAG, is a technique used to improve the responses of Large Language Models by providing them with relevant external information. Instead of depending only on the information learned during model training, a RAG system retrieves relevant information from a knowledge source and provides that information to the language model as context. The model then generates an answer based on the retrieved context.

A typical RAG pipeline contains several stages. First, a document is loaded into the system. The document is then divided into smaller pieces called chunks. Each chunk is converted into a numerical representation called an embedding. These embeddings are stored in a vector database. When a user asks a question, the question is also converted into an embedding. The system compares the question embedding with stored document embeddings and retrieves the most relevant chunks. These retrieved chunks are then provided to the language model as context, allowing the model to generate a more relevant answer.

ChromaDB is a vector database that can be used to store and search vector embeddings. In a RAG application, ChromaDB can store document chunks along with their embeddings and metadata. When a user asks a question, the system can perform a similarity search and retrieve the chunks that are most similar to the question. This makes ChromaDB useful for building simple and practical RAG applications.

Embeddings are numerical representations of text. Text with similar meanings tends to produce embeddings that are closer together in vector space. For example, the sentences "FastAPI is a Python framework" and "FastAPI is used to build Python APIs" have different words but similar meanings, so an embedding model can represent them as vectors with relatively high semantic similarity.

Chunking is an important part of a RAG pipeline. Large documents are divided into smaller chunks because sending an entire large document to a language model may be inefficient and may contain unnecessary information. Chunk size determines approximately how much text is included in each chunk, while chunk overlap allows some information from one chunk to be repeated in the next chunk. Overlap can help preserve context when important information occurs near the boundary between two chunks.

Recursive chunking is a commonly used approach for splitting documents. A recursive text splitter attempts to divide text using natural boundaries before moving to smaller boundaries when necessary. For example, it may first try to split text by paragraphs, then lines, then spaces, and finally individual characters. This approach can create more meaningful chunks than simply cutting text at an arbitrary character position.

Python, FastAPI, Generative AI, embeddings, vector databases, and RAG are commonly combined to create modern AI-powered applications. A typical application may use FastAPI as the backend API, Python for application logic, an embedding model for converting text into vectors, ChromaDB for similarity search, and a Large Language Model for generating the final response.